STAT 417 Final Code Work

Group 8

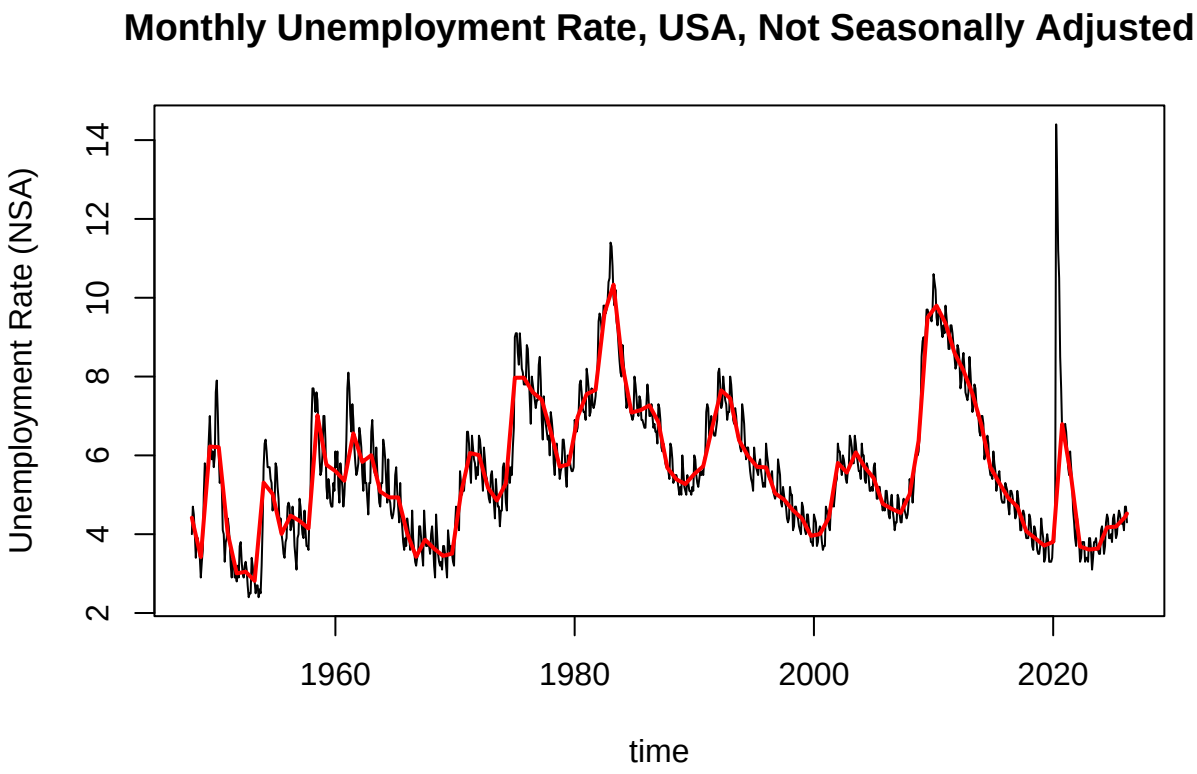
2026-04-26

Initial data pull/clean

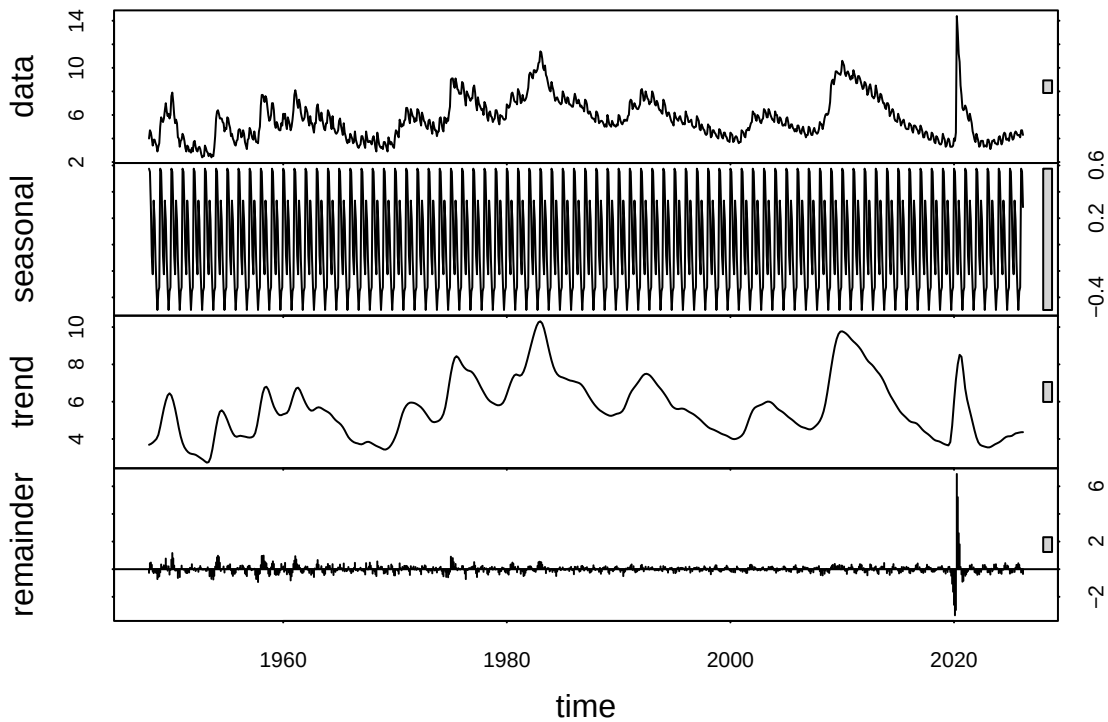
```
#read the data from the fed website
library(readr)
url <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATENSA"
unrate_data <- read_csv(url)

#convert to time series
unrate_ts <- ts(unrate_data$UNRATENSA, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unrate_ts_interp <- na.interp(unrate_ts)

#Seasonally adjusted data (for comparison maybe)
urlb <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATE"
unrate_data_sa <- read_csv(urlb)

#convert to time series
unratesa_ts <- ts(unrate_data_sa$UNRATE, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unratesa_ts_interp <- na.interp(unratesa_ts)
```

ts analysis

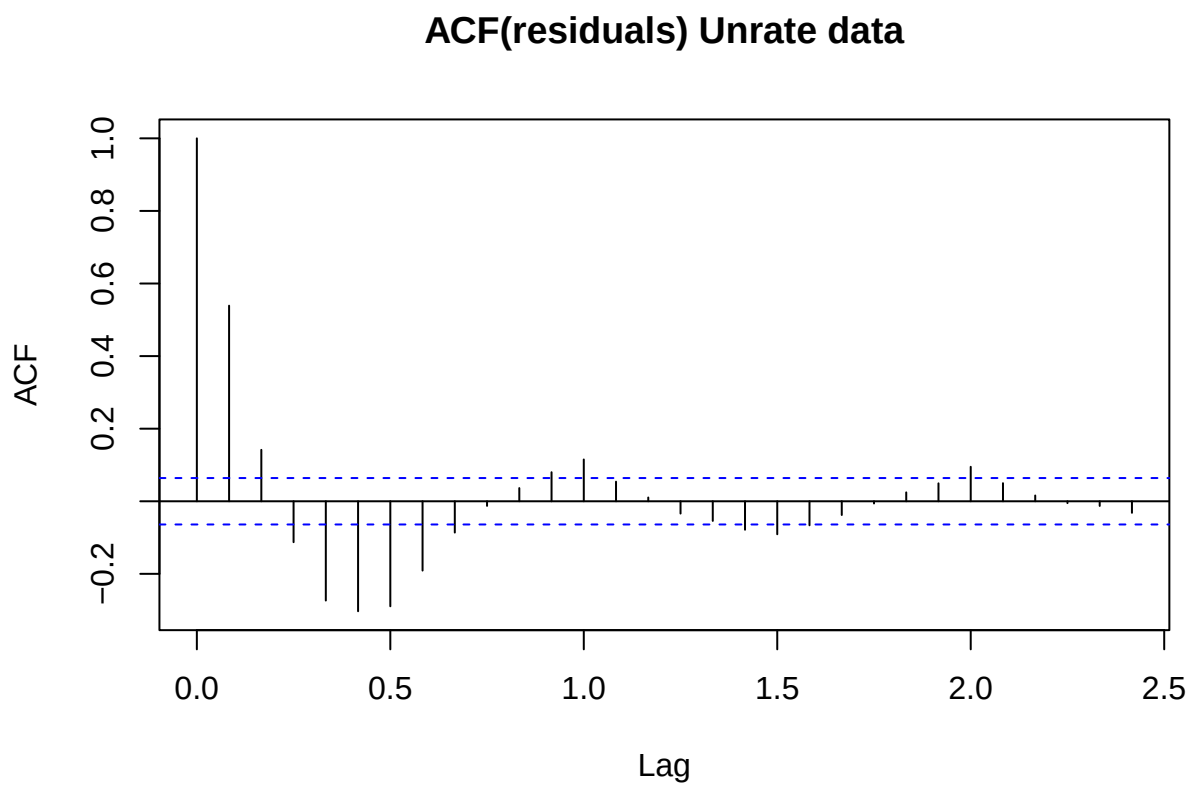
```
#full data set and smoothing
plot(unrate_ts_interp, ylab="Unemployment Rate (NSA)", xlab = "time", col = "black", main = "Monthly Un
lines(lowess(unrate_ts_interp, f=0.01), lwd=2, col="red")
```




```r
#stl analysis
unrate_stl <- stl(unrate_ts_interp, s.window = "periodic")
plot(unrate_stl)
```




```
season <- unrate_stl$time.series[,1]
trend <- unrate_stl$time.series[,2]
res <- unrate_stl$time.series[,3]

acf(res, main = "ACF(residuals) Unrate data")
```



We can see seasonality in the tiny spikes, probably should blow this graph up on the ppt.

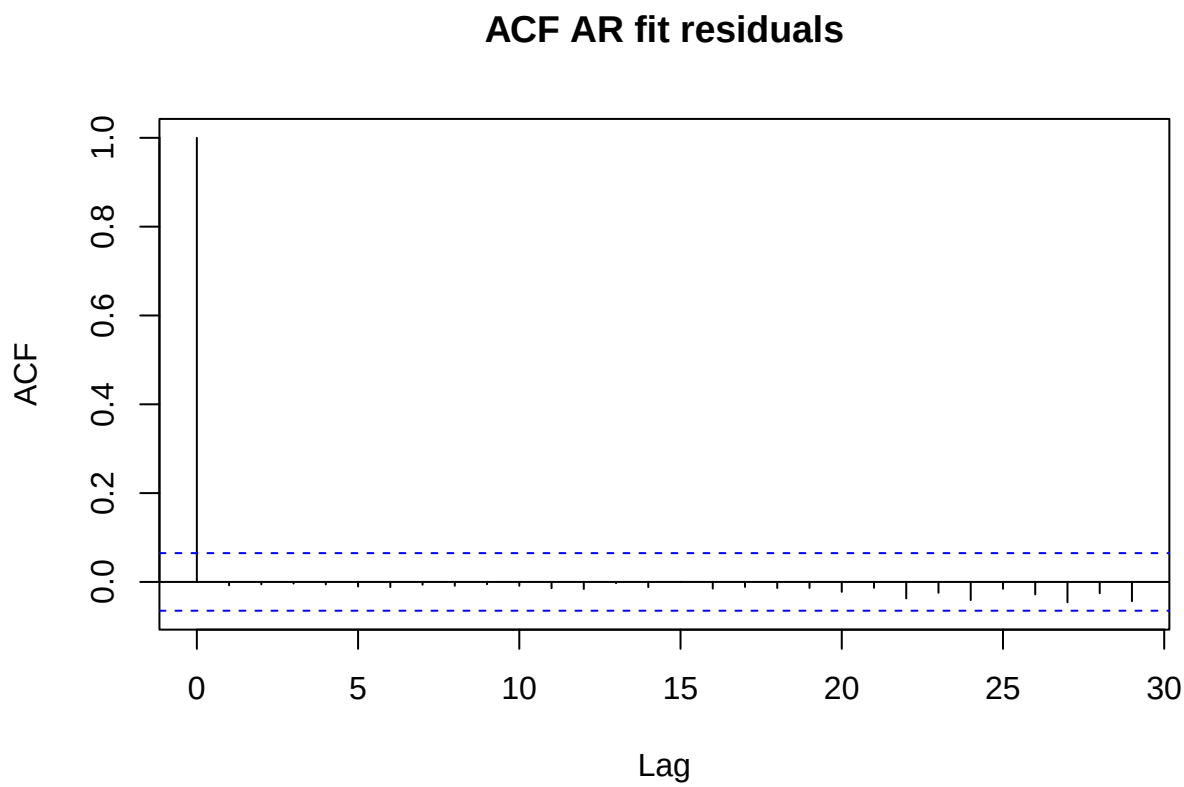
ACF reveals time dependence of the residuals, so we need to model the residuals.

Residual AR fit model

```
#get the order of the AR fit
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
#look at the residuals for this new model
fit_ar <- ar.yw(res, order.max = NULL)
fit_resid <- ts(fit_ar$resid)
p<- fit_ar$order
acf(fit_resid[-c(1:p)], main = "ACF AR fit residuals") #remove the first p(order) observations, the res
```



Trend model

```r
time <- 1:length(trend)
dat.trend <- data.frame(trend = trend, time = time)

# trig fit
fit_trig <- lm(trend ~
                 cos(2*pi*time/12) + sin(2*pi*time/12) +
                 cos(4*pi*time/12) + sin(4*pi*time/12) +
                 cos(6*pi*time/12) + sin(6*pi*time/12),
               data = dat.trend)
summary(fit_trig)
```

```
##
## Call:
## lm(formula = trend ~ cos(2 * pi * time/12) + sin(2 * pi * time/12) +
##     cos(4 * pi * time/12) + sin(4 * pi * time/12) + cos(6 * pi *
```

```
##     time/12) + sin(6 * pi * time/12), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9156 -1.2961 -0.1835  0.9681  4.6620
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            5.6601717  0.0523184 108.187   <2e-16 ***
## cos(2 * pi * time/12) -0.0046277  0.0740284  -0.063    0.950
## sin(2 * pi * time/12) -0.0093150  0.0739502  -0.126    0.900
## cos(4 * pi * time/12)  0.0034942  0.0739891   0.047    0.962
## sin(4 * pi * time/12) -0.0058515  0.0739893  -0.079    0.937
## cos(6 * pi * time/12)  0.0034437  0.0740282   0.047    0.963
## sin(6 * pi * time/12) -0.0007344  0.0739496  -0.010    0.992
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.603 on 932 degrees of freedom
## Multiple R-squared:  3.294e-05,  Adjusted R-squared:  -0.006405
## F-statistic: 0.005117 on 6 and 932 DF,  p-value: 1
```

```r
# polynomial fits
#I() forces literal interpretation of the eqn
fit_poly10 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) +

fit_poly9 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) + I

fit_poly8 <- lm(trend ~ time + I(time^2) + I(time^3) + I(time^4) + I(time^5) + I(time^6) + I(time^7) + I

summary(fit_poly10)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8) + I(time^9) +
##     I(time^10), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.1209 -0.8791 -0.1588  0.6790  4.0306
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.484e+00  4.151e-01  13.212  < 2e-16 ***
## time        -8.840e-02  3.036e-02  -2.912  0.00368 **
## I(time^2)    1.500e-03  7.276e-04   2.061  0.03956 *
## I(time^3)   -6.109e-06  8.115e-06  -0.753  0.45177
## I(time^4)   -3.030e-08  4.986e-08  -0.608  0.54359
## I(time^5)    3.462e-10  1.840e-10   1.881  0.06025 .
## I(time^6)   -1.272e-12  4.249e-13  -2.993  0.00283 **
## I(time^7)    2.420e-15  6.180e-16   3.916 9.65e-05 ***
## I(time^8)   -2.559e-18  5.499e-19  -4.653 3.75e-06 ***
## I(time^9)    1.426e-21  2.732e-22   5.221 2.20e-07 ***
```

```
## I(time^10)  -3.276e-25  5.806e-26  -5.643 2.22e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.12 on 928 degrees of freedom
## Multiple R-squared:  0.5144, Adjusted R-squared:  0.5092
## F-statistic: 98.31 on 10 and 928 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly9)
```

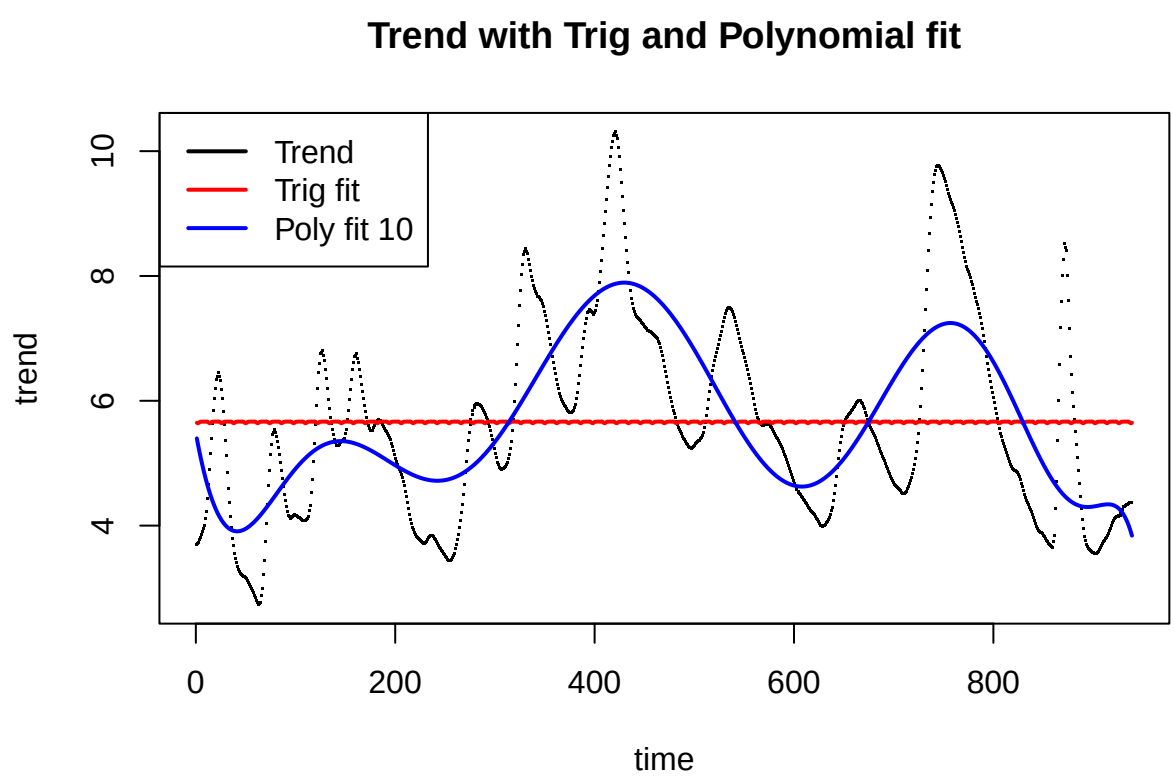
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8) + I(time^9),
##     data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.5979 -0.8635 -0.1588  0.6841  3.9585
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.486e+00  3.814e-01  17.006  < 2e-16 ***
## time        -2.023e-01  2.305e-02  -8.779  < 2e-16 ***
## I(time^2)    4.742e-03  4.537e-04  10.453  < 2e-16 ***
## I(time^3)   -4.580e-05  4.114e-06 -11.133  < 2e-16 ***
## I(time^4)    2.277e-07  2.023e-08  11.254  < 2e-16 ***
## I(time^5)   -6.404e-10  5.835e-11 -10.975  < 2e-16 ***
## I(time^6)    1.059e-12  1.015e-13  10.434  < 2e-16 ***
## I(time^7)   -1.018e-15  1.046e-16  -9.739  < 2e-16 ***
## I(time^8)    5.270e-19  5.876e-20   8.969  < 2e-16 ***
## I(time^9)   -1.134e-22  1.387e-23  -8.176 9.59e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.138 on 929 degrees of freedom
## Multiple R-squared:  0.4978, Adjusted R-squared:  0.4929
## F-statistic: 102.3 on 9 and 929 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly8)
```

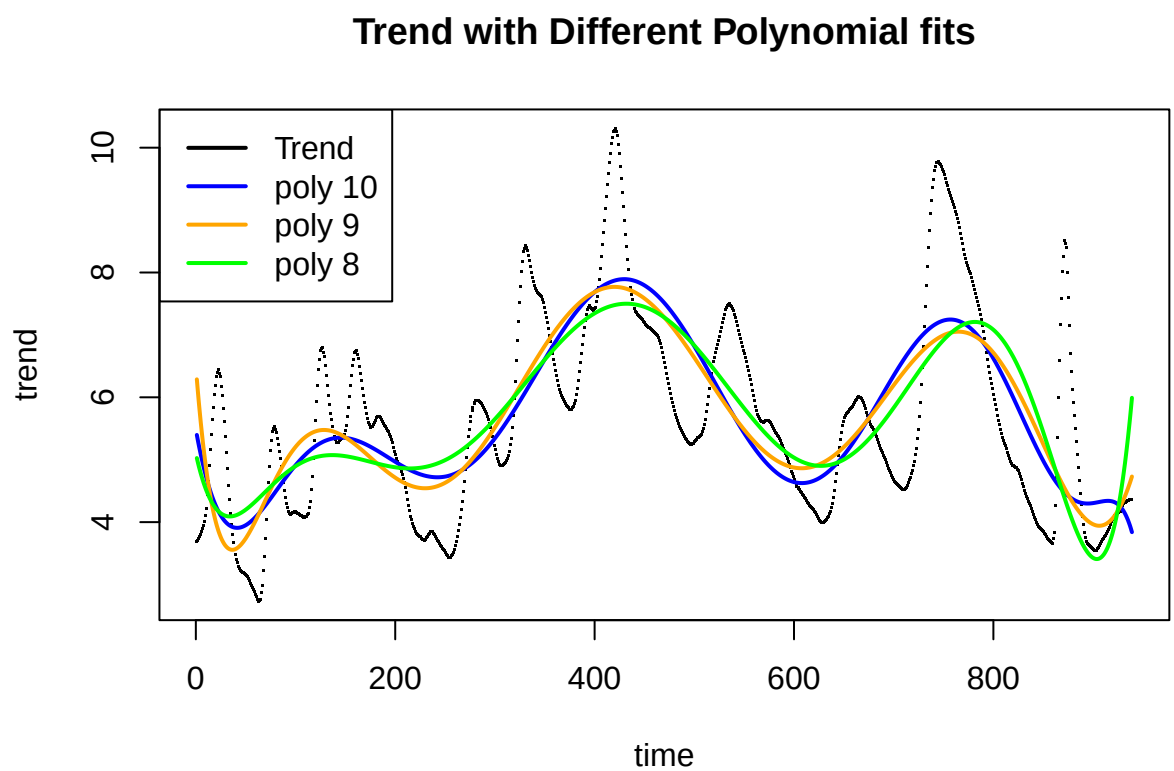
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7865 -0.9008 -0.0317  0.6954  4.1089
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.097e+00  3.534e-01  14.425  < 2e-16 ***
```

```
## time        -7.246e-02  1.728e-02  -4.194 3.01e-05 ***
## I(time^2)    1.725e-03  2.731e-04   6.317 4.12e-10 ***
## I(time^3)   -1.594e-05  1.960e-06  -8.132 1.34e-15 ***
## I(time^4)    7.314e-08  7.463e-09   9.800  < 2e-16 ***
## I(time^5)   -1.805e-10  1.608e-11 -11.231  < 2e-16 ***
## I(time^6)    2.437e-13  1.966e-14  12.399  < 2e-16 ***
## I(time^7)   -1.694e-16  1.271e-17 -13.323  < 2e-16 ***
## I(time^8)    4.735e-20  3.374e-21  14.033  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.178 on 930 degrees of freedom
## Multiple R-squared:  0.4616, Adjusted R-squared:  0.457
## F-statistic: 99.68 on 8 and 930 DF,  p-value: < 2.2e-16
```

```r
# compare trig vs polynomial
#fitted() applies the coefficients to the model for each data point
plot(time, trend, pch=".", main="Trend with Trig and Polynomial fit")
lines(time, fitted(fit_trig), col="red", lwd=2)
lines(time, fitted(fit_poly10), col="blue", lwd=2)
legend("topleft",
       legend=c("Trend", "Trig fit","Poly fit 10"),
       col=c("black", "red", "blue"),
       lwd=2)
```



```r
# compare polynomial degrees
plot(time, trend, pch=".", main="Trend with Different Polynomial fits")
lines(time, fitted(fit_poly10), col="blue", lwd=2)
lines(time, fitted(fit_poly9), col="orange", lwd=2)
lines(time, fitted(fit_poly8), col="green", lwd=2)
legend("topleft",
       legend=c("Trend", "poly 10","poly 9","poly 8"),
       col=c("black", "blue", "orange","green"),
       lwd=2)
```



Trig fit is not great, the plot is not even close and the p-values are high. Polynomial fit is good: of the three, degree 10 has the lowest residual standard error.

Seasonal model

```r
time <- 1:length(season)
dat.season <- data.frame(season = season, time = time)

fit_season1 <- lm(season ~
                    cos(2*pi*time/12) + sin(2*pi*time/12),
                  data = dat.season)

summary(fit_season1)
```

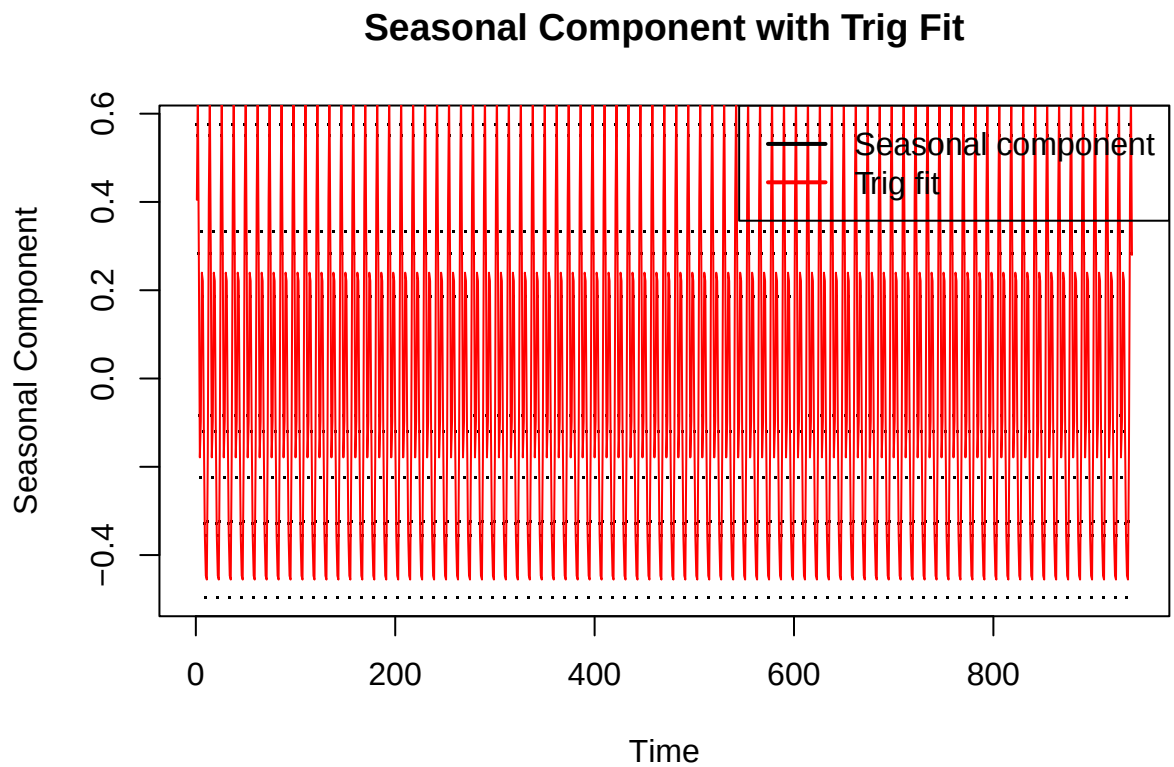
```
##
```

```
## Call:
## lm(formula = season ~ cos(2 * pi * time/12) + sin(2 * pi * time/12),
##     data = dat.season)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41997 -0.20964 -0.01918  0.29451  0.46633
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.0008124  0.0092394   0.088 0.929949
## cos(2 * pi * time/12) -0.0495083  0.0130733  -3.787 0.000162 ***
## sin(2 * pi * time/12)  0.3026840  0.0130595  23.177  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2831 on 936 degrees of freedom
## Multiple R-squared:  0.3706, Adjusted R-squared:  0.3693
## F-statistic: 275.6 on 2 and 936 DF,  p-value: < 2.2e-16
```

```r
fit_season2 <- lm(season ~
                  cos(2*pi*time/12) + sin(2*pi*time/12) +
                  cos(4*pi*time/12) + sin(4*pi*time/12) +
                  cos(6*pi*time/12) + sin(6*pi*time/12),
                data = dat.season)

summary(fit_season2)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * time/12) + sin(2 * pi * time/12) +
##     cos(4 * pi * time/12) + sin(4 * pi * time/12) + cos(6 * pi *
##     time/12) + sin(6 * pi * time/12), data = dat.season)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.157399 -0.081057  0.004208  0.094241  0.170761
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            6.604e-05  3.211e-03   0.021 0.983597
## cos(2 * pi * time/12) -5.051e-02  4.544e-03 -11.117  < 2e-16 ***
## sin(2 * pi * time/12)  3.016e-01  4.539e-03  66.453  < 2e-16 ***
## cos(4 * pi * time/12)  3.671e-02  4.541e-03   8.084 1.94e-15 ***
## sin(4 * pi * time/12)  3.400e-01  4.541e-03  74.875  < 2e-16 ***
## cos(6 * pi * time/12) -1.525e-01  4.544e-03 -33.559  < 2e-16 ***
## sin(6 * pi * time/12) -1.513e-02  4.539e-03  -3.333 0.000894 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0984 on 932 degrees of freedom
## Multiple R-squared:  0.9243, Adjusted R-squared:  0.9238
## F-statistic:  1897 on 6 and 932 DF,  p-value: < 2.2e-16
```

```r
plot(time, season, pch=".", main="Seasonal Component with Trig Fit",
     xlab="Time", ylab="Seasonal Component")
lines(time, fitted(fit_season2), col="red")
legend("topright", legend=c("Seasonal component", "Trig fit"),
       col=c("black", "red"), lwd=2)
```

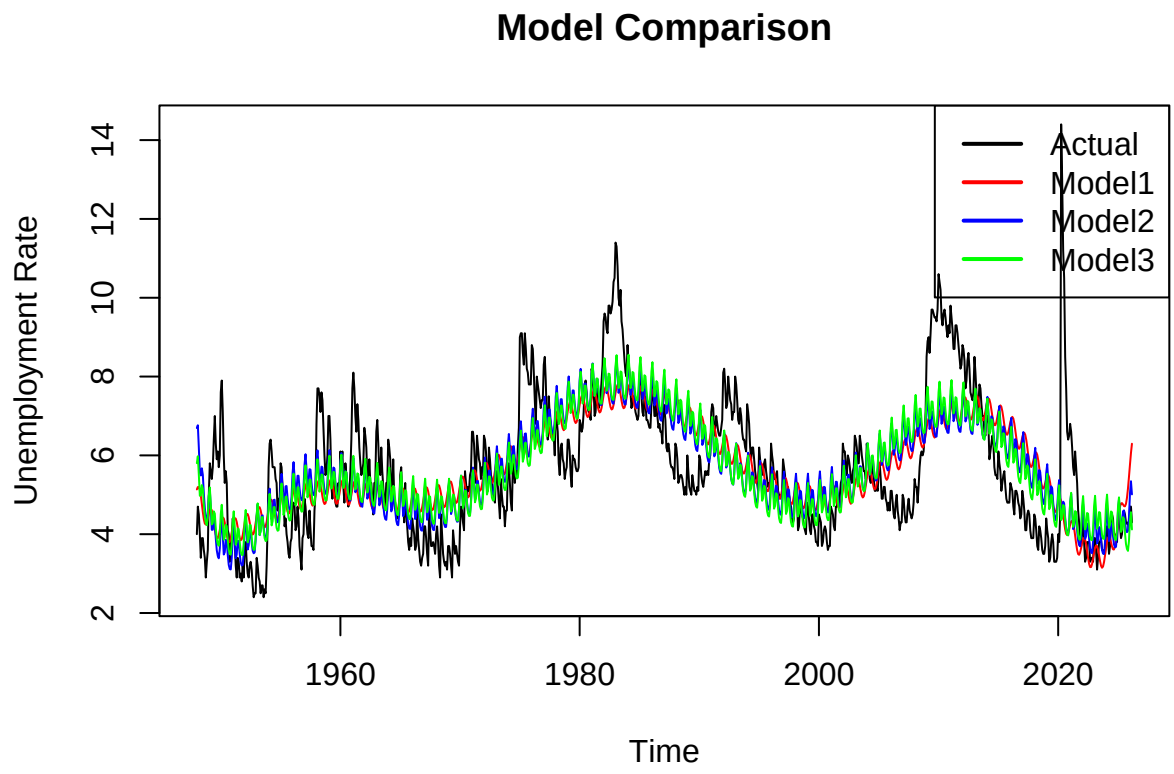


Graph is super ugly and compressed, but the more complex trig has higher R squared.

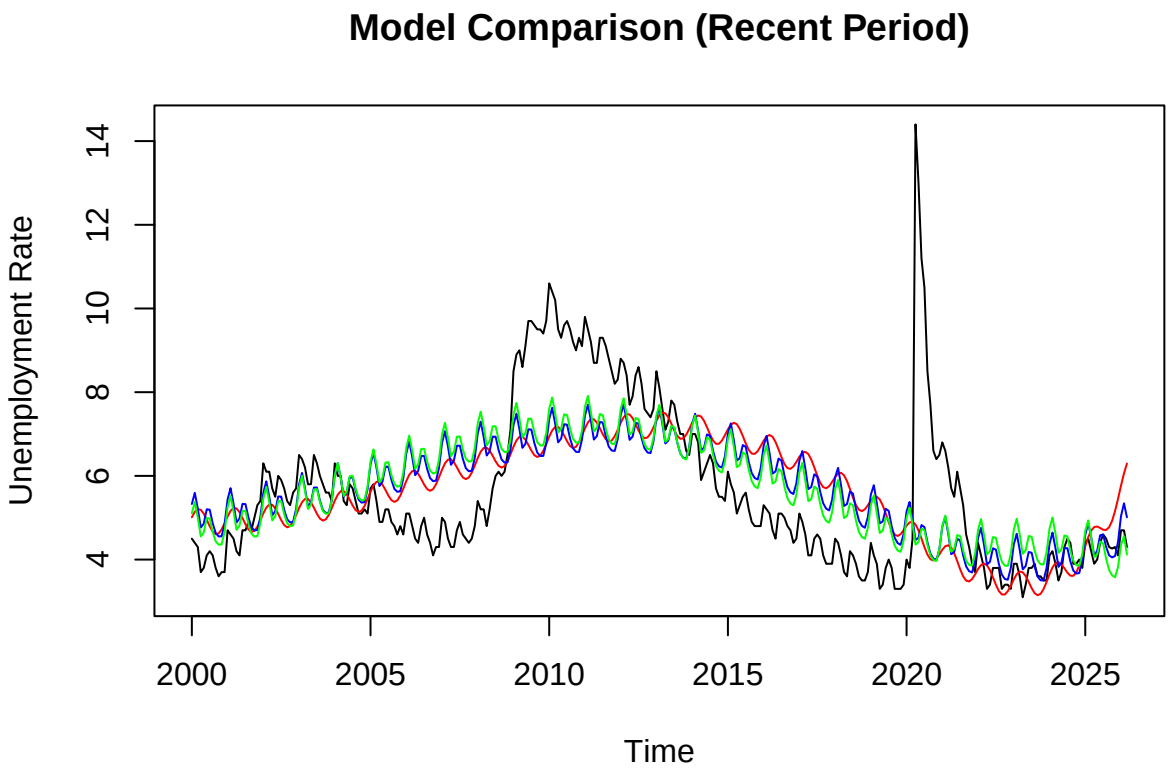
Combined Models

```r
#model 1: poly 8 and simple trig
tr1 <- fitted(fit_poly8)

coef_s1 <- coef(fit_season1)

ss1 <- coef_s1[1] +
       coef_s1[2]*cos(2*pi*time/12) +
       coef_s1[3]*sin(2*pi*time/12)

yhat1 <- tr1 + ss1

#model 2: poly 9 and complex trig
tr2 <- fitted(fit_poly9)
```

```r
coef_s2 <- coef(fit_season2)

ss2 <- coef_s2[1] +
       coef_s2[2]*cos(2*pi*time/12) +
       coef_s2[3]*sin(2*pi*time/12) +
       coef_s2[4]*cos(4*pi*time/12) +
       coef_s2[5]*sin(4*pi*time/12) +
       coef_s2[6]*cos(6*pi*time/12) +
       coef_s2[7]*sin(6*pi*time/12)

yhat2 <- tr2 + ss2

#model 3: ploy 10 and complex trig
tr3 <- fitted(fit_poly10)

yhat3 <- tr3 + ss2

#plot the models
plot(unrate_ts_interp, col="black",
     main="Model Comparison",
     ylab="Unemployment Rate")

lines(ts(yhat1, start=start(unrate_ts_interp), frequency=12),
      col="red")

lines(ts(yhat2, start=start(unrate_ts_interp), frequency=12),
      col="blue")

lines(ts(yhat3, start=start(unrate_ts_interp), frequency=12),
      col="green")

legend("topright",
       legend=c("Actual", "Model1", "Model2", "Model3"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```




```r
#only recent years
start_plot <- c(2000,1)

plot(window(unrate_ts_interp, start=start_plot), col="black",
     main="Model Comparison (Recent Period)",
     ylab="Unemployment Rate")

lines(window(ts(yhat1, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="red")

lines(window(ts(yhat2, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="blue")

lines(window(ts(yhat3, start=start(unrate_ts_interp), frequency=12), start=start_plot),
      col="green")
```


Model Comparison (Recent Period)
Unemployment Rate
4
6
8
10
12
14
2000
2005
2010
2015
2020
2025
Time